

AI Agents Setup Guide

Everything a new team member needs to get Claude Code, Codex, and Pi working with this repo's skills, tools, and optimizations. Follow in order.

What this repo is

A shared library of AI agent skills, tools, standards, and automation scripts. Your projects don't copy from it — they import from it via live @-references. When this repo is updated, your projects pick up changes automatically on the next session.

canon/	← this repo (shared library, clone once)
skills/	← sprint, pdf (catalog); wrapup, capture (deps
– loaded via sprint)	
tools/	← handoff, ticket, tkt.sh (infrastructure –
loaded via sprint)	
standards/	← efficiency (auto-injected into every
project)	
scripts/	← hook automation (auto-handoff, handoff-
inject, pre-commit)	
guides/	← this file
extensions/pi/	← Pi lifecycle extensions
your-project/	← your work repo
CLAUDE.md	← @-imports pointing into canon
AGENTS.md	← skill table + inlined standards for Codex
and Pi	
HANDOFF.md	← session context (auto-managed)

Setup

Step 1 — Clone canon

```
git clone https://github.com/sunitghub/canon.git ~/Developer/canon
export SKILLS=~/Developer/canon
ls $SKILLS/skills.sh # should exist
```

Step 2 — Install prerequisites

RTK (optional, recommended) — filters verbose CLI output before it hits the AI’s token budget. Saves 60–90% of tokens on common operations.

```
brew install rtk          # macOS
cargo install rtk        # WSL / Linux
rtk --version            # verify
```

If `rtk gain` fails after install, you likely have the wrong package (name collision on crates.io). Use `brew install rtk`, not `cargo install rtk`.

tk — bundled minimal ticket tool, no install needed. `skills add sprint` adds it and offers to put it on your PATH.

Step 3 — Agent setup (automated)

Claude Code only:

```
$SKILLS/skills.sh init
```

Re-run any time you move or rename the canon folder — it rewires hook paths in `~/.claude/settings.json`.

Claude Code + Codex + Pi:

```
$SKILLS/init-agent.sh      # interactive
$SKILLS/init-agent.sh claude # or pick one
$SKILLS/init-agent.sh all  # all three
```

Agent	What gets configured
Claude Code	Handoff + quality hooks merged into <code>~/.claude/settings.json</code> . RTK wired automatically if installed.
Codex	<code>~/.codex/RTK.md</code> + @reference in <code>~/.codex/AGENTS.md</code> (skipped if RTK absent).
Pi	Copies <code>extensions/pi/handoff.ts</code> to <code>~/.pi/agent/extensions/</code>

Manual fallback — Claude Code

```
rtk init -g --auto-patch # RTK native hook
```

Then merge into `~/.claude/settings.json` (replace `<SKILLS>` with your clone path):

```
{
  "hooks": {
    "Stop": [{ "matcher": "", "hooks": [{ "type":
      "command", "command": "<SKILLS>/scripts/auto-
handoff.sh" }] }],
```

```

    "UserPromptSubmit": [{ "matcher": "", "hooks": [{ "type":
      "command", "command": "<SKILLS>/scripts/handoff-
      inject.sh" }] }],
    "PostToolUse":      [{ "matcher": "Bash", "hooks": [{ "type":
      "command", "command": "<SKILLS>/scripts/auto-polish-
      trigger.sh" }] }],
    "PreToolUse":       [{ "matcher": "Bash", "hooks": [{ "type":
      "command", "command": "<SKILLS>/scripts/pre-commit-
      check.sh" }] }]}
  }
}

```

Step 4 — Per-project setup

Run once per project:

```

cd /path/to/your-project
$SKILLS/skills.sh addall      # register all available skills
                              (recommended)

# Or pick individually:
$SKILLS/skills.sh add sprint  # full dev workflow (includes
                              everything)
$SKILLS/skills.sh add pdf     # PDF read/extract/merge/split

```

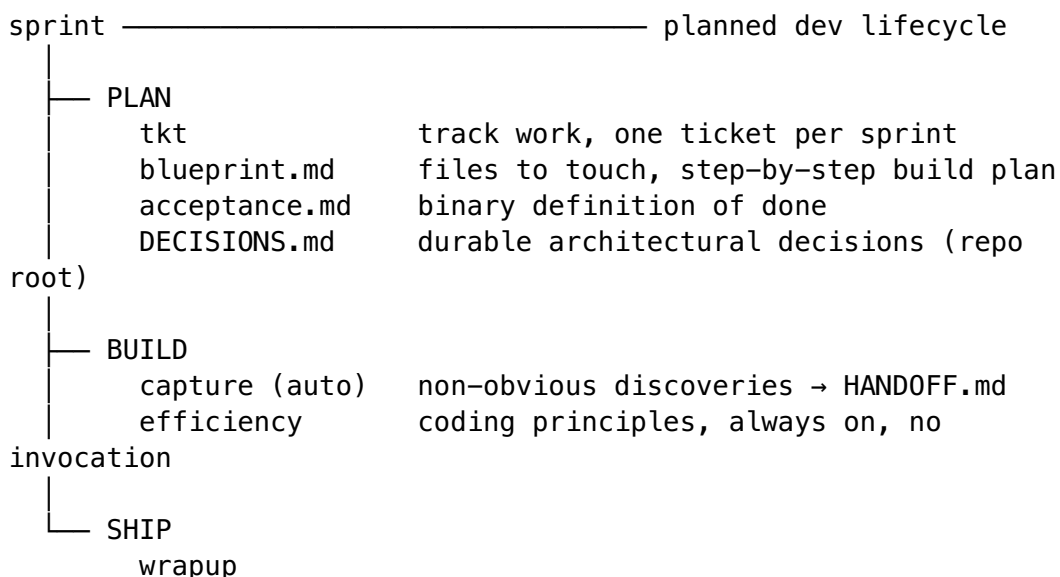
Then verify:

```
$SKILLS/skills.sh status
```

To initialize HANDOFF.md, tell the agent: *“Initialize the handoff file.”*

What you get — and why it’s built this way

Installing sprint gives you a full dev lifecycle in two commands. Everything else is automatic. Here’s the full picture:



code-simplifier	clarity and redundancy pass
code-reviewer	seven-dimension logic review
security-review	high-confidence vulnerability scan

Session hooks (fire automatically – no commands needed):

handoff-inject	session start → agent reads HANDOFF.md silently
auto-handoff	session end → agent snapshots git state to HANDOFF.md

The layers below explain each component from the ground up — what pain it solves and how it works.

Layer 1 — Session continuity

The problem: AI agents start cold. Every new session — or context window exhaustion mid-session — means re-explaining where things stand. Over a long project, this overhead compounds.

What it does: HANDOFF.md is a git-tracked file in the project root. It holds current focus, in-progress work, recent decisions, and mid-session discoveries. Two hooks automate its lifecycle:

- **handoff-inject** (fires on session start) — injects HANDOFF.md silently into your first prompt, once per 4-hour window. The agent wakes up knowing where things stand without you saying a word.
- **auto-handoff** (fires on session end) — appends a timestamped snapshot: modified files, recent commits, active tickets. Safety net when context runs out mid-session.

What triggers it: Automatically. No commands. Between sessions, the agent reads and writes it as part of `sprint start` and `sprint complete`.

What goes in it:

Handoff

Current Focus

One sentence – what are we working on.

In Progress

– file/path or ticket-id: what's started but not finished

Recent Decisions

– Decision and WHY (not what – the diff shows what)

Discoveries

– Non-obvious facts found through investigation

Next Steps

1. First concrete next action

Layer 2 — Knowledge capture

The problem: Agents discover non-obvious constraints mid-session — a connection pool cap, a config file location, an edge case only found by running the code. Without recording them immediately, they're lost when context compacts or the session ends.

What it does: capture writes discoveries to `HANDOFF.md` `## Discoveries` the moment they're found — not at wrapup, not at session end. Each entry also saves a project memory so future sessions can recall it without reading the file.

Qualifies: - Filter or exclusion rules found through experimentation - Numerical facts not derivable from code (row counts, limits, offsets) - Environment gotchas — args, paths, config locations, build quirks - Architecture decisions with non-obvious WHY - Any constraint requiring active investigation that isn't visible in the code

What triggers it: Automatic — fires whenever the agent discovers something qualifying. To force-capture something:

Agent	Trigger
Claude Code	<code>/capture <text></code>
Codex / Pi	“Capture this” / “Record this in discoveries”

Layer 3 — Coding standards (always-on)

The problem: Every new session, the agent may drift from your project's conventions — import style, naming, git commit format, comment rules — without a reminder.

What it does: The efficiency standard is injected into every project that has any canon skill registered. It covers coding principles (simplicity first, surgical changes, no speculative features), git conventions (commit format, branch discipline), and token-efficiency rules. It's never shown in `skills list` and needs no invocation — it just runs.

Layer 4 — Code quality

Three focused passes, each with a clear job:

code-simplifier — clarity and redundancy pass. Reduces nesting, eliminates dead code, improves names, removes obvious comments. Never changes behavior — only how the code reads.

code-reviewer — seven-dimension logic review: correctness, maintainability, readability, efficiency, security, edge cases, test coverage. Reports as Critical / Improvements / Nitpicks / Recommendations.

security-review — high-confidence vulnerability scan. Traces data flow end-to-end before flagging anything. Only reports HIGH (confirmed vulnerable pattern + attacker-controlled input) or MEDIUM (pattern confirmed, input source unclear). No noisy pattern-match reports.

Each step has skip logic — it states why in one line when it doesn't apply:

Step	Skipped when
code-simplifier	Single-line change, or docs/config only
code-reviewer	Single-line fix with no design implications
security-review	No auth, DB, user input, API, crypto, or file I/O changed

Layer 5 — Wrapup

The problem: The three quality steps above need to run in a specific order (simplify first, then review clean code, then security on the reviewed version) and each has skip logic that has to be evaluated. Remembering to run them in sequence is friction.

What it does: wrapup runs all three steps in order, evaluates skip logic automatically, and reports a single structured summary. Inside `sprint complete`, it runs on all files modified since `sprint start`.

Outside a sprint: run `/wrapup` directly on any code written in the session.

Layer 6 — Sprint (the full lifecycle)

The problem: Planning, building, and shipping code each require different behaviors from the agent — upfront design before touching code, structured build execution, quality gates before closing. Invoking these separately means remembering what to run when, and it's easy to skip a step.

What it does: `sprint` encapsulates everything above into two commands:

Command	What happens
<code>sprint start</code>	Creates ticket → blueprint → acceptance criteria → reads DECISIONS.md + HANDOFF.md → produces sprint brief → waits for your approval
<code>sprint complete</code>	Runs wrapup on modified files → validates every acceptance criterion → appends decisions to DECISIONS.md → updates HANDOFF.md → closes ticket

Trigger phrases: - sprint start: *“sprint start”, “start a sprint for X”, “let’s work on X”* - sprint complete: *“sprint complete”, “approve”, “ship it”*

Planning files (colocated with the ticket):

```
.tickets/<id>/
  ticket.md      ← tkt-managed
  blueprint.md   ← files to touch, step-by-step build plan
  acceptance.md  ← binary definition of done
```

DECISIONS.md (repo root) — durable log of non-obvious architectural choices. Sprint start reads it; sprint complete writes to it. Separate from HANDOFF.md (session state) — this is the permanent record.

Decisions

```
| Date | Decision | Reason |
|----|----|----|
| 2026-05-17 | Amounts stored as integer cents | Avoid float
              precision bugs |
```

A complete session — from idea to shipped

Everything in **bold** is something you type. Everything else happens automatically.

Opening the session

You open Claude Code and type your first message. Before it reaches the agent:

handoff-inject fires — reads HANDOFF.md silently into your first prompt (once per 4-hour window).

The agent wakes up knowing the current project state, prior decisions, and any mid-session discoveries from the last session.

You: “Sprint start — add rate limiting to the login endpoint”

The agent:

1. Runs `tkt create "Add rate limiting to login endpoint"` → `ticket t-r4t3`
2. Runs `tkt start t-r4t3`
3. Creates `.tickets/t-r4t3/blueprint.md` and `.tickets/t-r4t3/acceptance.md`
4. Reads `DECISIONS.md` — finds: *“Redis chosen for session state”*
5. Reads `HANDOFF.md` — picks up any open blockers or prior context
6. Produces a sprint brief and waits:

Sprint `t-r4t3` – Add rate limiting to login endpoint

Goal: Block brute-force attempts by tracking failed logins per IP

in Redis.

Files to modify:

auth/views.py - add rate limit check to login endpoint
auth/middleware.py - new @rate_limit decorator

Files to create:

tests/test_rate_limit.py

Acceptance criteria:

- ✓ Login returns 429 after 5 failed attempts from the same IP
- ✓ Counter resets after 15 minutes
- ✓ Rate limit is not bypassable via header manipulation
- ✓ Tests pass

Constraint from DECISIONS.md: use Redis (already a dependency).

Ready to proceed?

You: “Yes”

During the build

The agent writes code. While reading the Redis client config, it notices the connection pool is set to 5 — which could cause rate limit checks to queue under heavy login load.

capture fires automatically: - Appends to HANDOFF.md under ##
Discoveries: “Redis connection pool capped at 5 — rate limit checks may queue under sustained login load. See config/redis.py:12.” - Saves a project memory.

You didn’t ask for this. It happened because the agent detected a non-obvious constraint.

You: “Sprint complete”

The agent:

1. **Wrapup pipeline** — runs on all files modified since sprint start:

- code-simplifier — removes a redundant null check in middleware.py, clarifies a variable name
- code-reviewer — flags a missing test for the header-manipulation bypass criterion
- security-review — confirms no injection risk in the Redis key construction

2. **Acceptance check** — reviews acceptance.md:

- ✓ 429 returned after 5 failures
- ✓ Counter resets after 15 minutes
- ✗ No test for header bypass (flagged by reviewer)

- ? Tests pass — uncertain, bypass test missing

Stops. Reports: *“Criterion 3 not met — bypass test is missing. Adding it now.”* Writes the test. Re-checks: all ✓.

3. DECISIONS.md — appends:

Date	Decision	Reason
2026-05-17	Rate limit key uses first IP from X-Forwarded-For	Prevents bypass via appended IPs while preserving proxy compatibility

4. HANDOFF.md — updates ## Next Steps: *“Consider alerting on repeated 429s — out of scope for this sprint.”*

5. Runs `tkl close t-r4t3`

6. Reports: *“Shipped. Rate limiter live on login endpoint. Redis pool cap captured in Discoveries. One decision recorded. Follow-up noted in HANDOFF.”*

Closing the session

You close Claude Code. Before it exits:

auto-handoff fires — appends a snapshot to HANDOFF.md: modified files, recent commits, active tickets.

Next session — or the next agent — reads the file and picks up exactly where this one left off.

Reference

Skills commands

```
$SKILLS/skills.sh list           # show available skills
$SKILLS/skills.sh add sprint     # register a skill in
    current project
$SKILLS/skills.sh addall         # register all skills
    (idempotent)
$SKILLS/skills.sh status         # check registration +
    freshness
$SKILLS/skills.sh refresh        # re-register + heal
    stale paths + prune covered deps
```

Ticket commands

Sprint manages the full lifecycle automatically. Use `tkl` directly for queries:

```
tkit ls                                # list all tickets
tkit ls --status=in_progress          # filter by status
tkit show <id>                        # full ticket detail
tkit reopen <id>                      # reopen a closed ticket
```

Need dependency tracking, tags, or assignees? Install ticket (brew install ticket) — same .tickets/ format, fully compatible.

Skill verification

Skill	How to verify	Expected response
sprint	"Start a sprint for X"	Sprint brief produced, awaits approval before any code
pdf	"Extract text from [file].pdf"	Extracted content, or a clear error
ticket	tkit ls	Empty list or existing tickets — no error

Everything else is automatic. efficiency is always on. capture fires mid-session. wrapup runs inside sprint complete. handoff and ticket are deps of sprint — loaded silently.

Staying updated

```
cd $SKILLS && git pull
```

Hook scripts update immediately — called by path, so the new version runs on the next session.

Skill content in CLAUDE.md is live @-import references — Claude picks up changes automatically on the next session.

Inline standards in AGENTS.md (Codex, Pi) are a static copy — refresh explicitly:

```
$SKILLS/skills.sh refresh /path/to/your-project
```

refresh re-registers every skill, replaces outdated standard blocks, heals stale @-import paths, and prunes covered deps — all in one command.

For newly added skills:

```
$SKILLS/skills.sh addall /path/to/your-project
```

Check for issues first:

```
$SKILLS/skills.sh status /path/to/your-project
```